ReSolvitaire Cache Redesign: Architecture and Payload Specification

Consolidated Design Document

Ian Gent AI Assistant (Claude, Anthropic)

March 2026

AI Generation Statement: *This document was produced collaboratively. The architectural decisions—unified single cache, no live bits, card-centric encoding, cells-in-descriptor, 4-bit nibble layout—emerged from iterative discussion between Gent and the AI. Key structural insights (starting-position covers stock/waste/reserve; root needs no pile identifier; false negatives on cycle detection are acceptable; encoding need not be reconstructible) are Gent's contributions. The literature survey, Zobrist hashing scheme, flat-table design analysis, and bit-level payload specification are the AI's contributions. Neither author guarantees correctness of all details; empirical validation is recommended before deployment.*

Contents

Executive Summary

We propose replacing Solvitaire's Boost.MultiIndex LRU cache with a flat open-addressed hash table using Zobrist hashing and a compact 32-byte per-entry payload. The design targets single-deck games without sequences or accordion; unsupported games fall back to the existing cache.

Cache structure. A single flat array of fixed-size entries. Zobrist hashing provides O(1) incremental hash updates; per-pile hashes are combined via modular addition for interchangeable piles, yielding symmetry-invariant indexing without sorting [2, 3]. No live bits, no LRU machinery, no separate cycle-detection structure. States are inserted on forward visit; evicted ancestors cause false negatives on cycle detection (redundant search), never false positives (incorrect pruning). Replacement policy is a free parameter [5, 6].

Payload. 32 bytes per entry: 16-bit depth counter, 16-bit foundation/hole encoding, 6-bit waste pointer, and 52 four-bit per-card descriptors. Each card's descriptor encodes its "situation" (starting position, in cell, root, built on parent i, in hole) without referencing pile indices, making the encoding inherently canonical under pile permutation. The payload is maintained incrementally during search (1–3 nibble writes per move), so cache insertion is a bare `memcpy`—no encoding computation on the critical path. Comparison uses `memcmp` over the 30 non-metadata bytes.

Compression. Estimated 3–7× reduction in per-entry size versus the current `cached_game_state` vector, yielding 3–7× more entries in the same memory. Combined with elimination of Boost.MultiIndex pointer overhead (24–48 bytes per entry), effective capacity increase is larger still.

Correctness. Zero false positives on transposition detection (full-state comparison on every hit). Zero false positives on cycle detection (same mechanism). False negatives possible on both (evicted entries), causing only redundant work. Mathematical correctness of winnability/unwinnability proofs is preserved.

Part I
Cache Architecture

1 Zobrist Hashing with Symmetry-Invariant Combining

1.1 Standard Zobrist Hashing

Pre-compute a table of random 64-bit values Z[card][pile_role][position]. The hash of a state is the XOR of all active entries. On `make_move`/`undo_move`, XOR out the old contribution and XOR in the new: O(1) per card moved [1].

1.2 Two-Layer Combining for Pile Symmetry

Layer 1: Each pile maintains its own Zobrist hash via XOR (standard).

Layer 2: The global hash combines per-pile hashes. Non-interchangeable piles (foundations, stock, waste, hole) contribute via XOR. Interchangeable piles (tableau, cells, unstacked reserve) contribute via **modular addition** (mod 2^{64}).

Modular addition is commutative and preserves multiset information ($A + A \neq 0$, unlike $A \oplus A = 0$), making the global hash invariant under permutation of interchangeable piles [2]. Incremental update when a card moves between interchangeable piles A and B: subtract old pile hashes, update both via XOR, add new pile hashes. Six constant-time operations [3].

1.3 Hash Indexing

The N-bit table index is derived from the Zobrist hash by masking (for power-of-2 tables) or `mul_hi64` (for arbitrary sizes, following Stockfish [9]). Remaining hash bits serve as a fast verification filter, though full-state comparison is the correctness guarantee.

2 Flat Open-Addressed Table

A single contiguous array of fixed-size entries (32 bytes each). No heap allocation, no linked lists, no Boost.MultiIndex.

Cluster design (recommended). Following Stockfish [8], allocate 64-byte cache-line-aligned clusters, each containing 2 entries ($2 \times 32 = 64$ bytes). On collision, probe both entries in the cluster; replace according to the chosen policy. This bounds probe cost to 1 cache-line load.

Replacement policy. A free parameter: always-replace, depth-preferred [7], age-weighted, or TwoBig1 [4]. No correctness dependency on the policy. Empirical tuning recommended.

Memory. At 32 bytes per entry with 2 entries per 64-byte cluster, a 4 GB allocation holds $\sim 6.7 \times 10^7$ clusters $= 1.34 \times 10^8$ entries. Compare to the current Boost.MultiIndex cache at ~100–120 bytes effective per entry: the same 4 GB holds $\sim 3.5 \times 10^7$ entries. **$\sim$3.8$\times$ capacity increase** from the data structure change alone, before accounting for improved cache-line utilisation.

3 Unified Single-Cache Design

3.1 No Separate Cycle Detection

Solvitaire's current design marks cached states as "live" (ancestors on the DFS path) and refuses to evict them. This is eliminated entirely.

Rationale: Correctness requires zero false positives (never claiming a novel state was visited). It does *not* require zero false negatives. A missed cycle detection (because the ancestor was evicted) causes the solver to re-explore a subtree—redundant work, not an incorrect answer.

3.2 Insert on Forward Visit

States are inserted when first encountered during forward DFS, not on backtrack. This ensures ancestors are in the cache during forward exploration, providing best-effort cycle detection as a side effect.

3.3 Safety Bound

A configurable depth limit guards against unbounded looping if an ancestor is evicted and a cycle is missed. Abort if depth exceeds a bound (e.g., cache capacity, or a game-specific maximum). Negligible overhead.

3.4 Correctness Summary

Property	Guarantee	Consequence of violation
Transposition false positive	Zero	Would invalidate proof
Transposition false negative	Possible	Redundant search
Cycle false positive	Zero	Would invalidate proof
Cycle false negative	Possible	Redundant search

4 Backward Compatibility

Both cache implementations (new flat table and existing Boost.MultiIndex LRU) are retained. Selection at construction time based on `sol_rules`: games with sequences, accordion, or two decks use the old cache; all others use the new cache. The solver interacts through a common interface (`insert`, `contains`, `clear`, `size`). The old cache's `set_non_live` and iterator-returning `insert` are wrapped behind this interface. As the new encoding is extended to more game types, the selection boundary shifts.

Part II
Payload Specification

5 Scope

Single-deck (52-card, max rank 13) games using any combination of: foundations, hole, tableau, stock, waste, reserve (stacked/unstacked), cells. Exactly one of foundations or hole is present (enforced by `rules_parser.cpp`). No sequences, no accordion. Streamliner setting: `NONE` (no suit symmetry).

6 Layout

32 bytes = 256 bits.

Field	Offset	Size	Description
Depth	bits 0–15	16 bits	Search depth (0–65534; 65535 = large)
Foundations	bits 16–31	16 bits	Foundation top ranks OR hole-top encoding
Waste-ptr	bits 32–37	6 bits	Waste pointer (0 if no stock)
Padding	bits 38–47	10 bits	Reserved, zero-filled
Descriptors	bits 48–255	208 bits	52 per-card descriptors, 4 bits each

Total: $16 + 16 + 6 + 10 + 208 = 256$ bits $= 32$ bytes. ✓

7 Field Specifications

7.1 Depth (bits 0–15)

16-bit unsigned integer. Values 0–65534: search depth at time of insertion. Value 65535: depth ≥ 65535. **Excluded from comparison**: cache hit verification compares bits 16–255 only (bytes 2–31). Available for replacement-policy decisions.

7.2 Foundations (bits 16–31)

Foundation games: Four 4-bit values encoding foundation top ranks. Suit order: Clubs (bits 16–19), Hearts (20–23), Spades (24–27), Diamonds (28–31). Value 0 = empty; 1 = Ace; ...; 13 = King.

Hole games: Bits 16–21 encode the hole top card as a 6-bit card ID (Section 7.5). Bits 22–31 are zero. The sets of cards in the hole vs. on tableau are determined by the per-card descriptors; only the hole *top* needs separate encoding (it determines move legality).

The two uses are mutually exclusive (exactly one of hole/foundations per game).

Any card at or below its suit's foundation top is "on foundations." Its per-card descriptor is set to Starting (0) and is not read during comparison.

7.3 Waste-ptr (bits 32–37)

6-bit index (0–63) encoding the deal pointer within the stock/waste system. For games without stock: zero. All single-deck games have stock size $\leq 50 < 64$.

The combined stock/waste/reserve state is fully determined by: (a) the initial deal (known), (b) which cards have left these systems (encoded by non-Starting descriptors), and (c) this pointer. No per-card bitmask is needed for stock, waste, or reserve cards.

7.4 Descriptors (bits 48–255, 208 bits)

52 four-bit nibbles, one per card, in fixed card ID order (Section 7.5).

Value	Name	Meaning
0	STARTING	Card is in its initial deal position (face-down/face-up tableau, stock, waste, reserve, pre-filled cell). Also used for cards on foundations (descriptor ignored).
1	IN_HOLE	Card has been played to the hole (hole games only).
2	ROOT	Bottom face-up card of a tableau pile, not in starting position (placed in a space or revealed after built cards above were moved).
3	IN_CELL	Card is in a cell.
4	PARENT_0	Built on first legal parent (see Section 7.6).
5	PARENT_1	Built on second legal parent.
6	PARENT_2	Built on third legal parent.
7	PARENT_3	Built on fourth legal parent.
8–15	RESERVED	Zero. Available for future use (two-deck parents, streamliner flags, etc.).

Design notes.

- IN_CELL (value 3) replaces the dedicated cell encoding from the earlier specification. Since cells are interchangeable, "in a cell" is a pile-index-free descriptor. No separate cell-card list or sorting needed.
- IN_HOLE (value 1) and foundation-top encoding share the FOUNDATIONS field (Section 7.2), exploiting their mutual exclusivity.
- Values 0–3 are fixed across all game types. Values 4–7 are parent slots whose meaning depends on `build_pol`. Values 8–15 are reserved.
- Max-rank cards (kings) have no parents. Their descriptors use only 0–3.
- When a card moves to foundations, its descriptor is set to 0 (STARTING) to ensure `memcmp` correctness.
- Cards dealt from stock to tableau in stock-to-all-tableau games (e.g. Spider variants) that don't match the card below them are encoded as ROOT.

7.5 Card ID Mapping

card_id = suit × 13 + (rank − 1). Suits: Clubs=0, Hearts=1, Spades=2, Diamonds=3. Range: 0 (Ace of Clubs) to 51 (King of Diamonds). Matches Solvitaire's `card::suit` and rank conventions.

7.6 Parent Ordering

For a card of rank r and suit s, its legal parents (rank $r+1$) are indexed by suit of the parent card in fixed order:

`RED_BLACK`: Two opposite-colour parents, ordered by parent suit index. E.g., a red card's parents are the $(r+1)$ of Clubs (PARENT_0) and Spades (PARENT_1).

`SAME_SUIT`: One parent: $(r+1)$ of the same suit. Always PARENT_0.

`ANY_SUIT`: Four parents: $(r+1)$ of Clubs, Hearts, Spades, Diamonds = PARENT_0 through PARENT_3.

`NO_BUILD`: No parents. Cards use only STARTING, ROOT, IN_CELL, IN_HOLE.

8 Nibble Access

With 4-bit descriptors packed two per byte, access is simple and never spans byte boundaries.

For card c $(0 \le c \le 51)$:

```
byte_idx = 6 + (c / 2);       // descriptors start at byte 6
if (c % 2 == 0) {             // low nibble
    value = payload[byte_idx] & 0x0F;
} else {                       // high nibble
    value = (payload[byte_idx] >> 4) & 0x0F;
}
```

Writing:

```
if (c % 2 == 0) {
    payload[byte_idx] = (payload[byte_idx] & 0xF0) | (new_val & 0x0F);
} else {
    payload[byte_idx] = (payload[byte_idx] & 0x0F) | (new_val << 4);
}
```

One read-modify-write per card. No byte-boundary spanning.

9 Incremental Maintenance

The payload is a `uint8_t payload[32]` maintained alongside the `game_state`.

9.1 Initialisation

Zero all 32 bytes. Set FOUNDATIONS to initial top ranks (or hole-top card ID). Set WASTE-PTR to initial value. All descriptors are 0 (STARTING): correct, since every card is in its starting position.

9.2 Update Cost per Move Type

Move type	Operations
Regular (card to tableau parent)	1 nibble write
Regular (card to foundation)	1 nibble write + foundation field update
Regular (card to cell)	1 nibble write
Regular (card to hole)	1 nibble write + hole-top update
Built group move	1 nibble write (bottom card of group only)
Reveal face-down card	0 (stays STARTING)
Stock-to-waste deal	1 field write (waste pointer)
Stock-k-plus move	1–2 writes (waste pointer + played card)
Stock-to-all-tableau	T nibble writes (T = tableau pile count)

Each nibble write is a single byte read-modify-write. Undo restores the previous nibble value (stored on the move stack: 4 bits per affected card).

9.3 Cache Operations

Insertion: `memcpy(entry, payload, 32)`. No encoding computation.

Comparison: `memcmp(entry + 2, payload + 2, 30)`. Skip first 2 bytes (depth).

Total critical-path cost per cache probe: One Zobrist-derived index computation (already maintained) + one 30-byte `memcmp` + one conditional 32-byte `memcpy`.

10 Canonicality

No descriptor references a pile index:

- STARTING: "in initial deal position."
- ROOT: "bottom of some tableau pile, not in starting position."
- IN_CELL: "in some cell."
- PARENT_i: "built on card X" (card identity, not pile position).
- IN_HOLE: "in the hole."

Cards are enumerated in fixed ID order. Permuting interchangeable piles changes no descriptor. Combined with the Zobrist hash's symmetry invariance (additive combining), the entire system handles pile symmetry purely by construction. No sorting at any stage.

11 Injectivity Argument

Claim: Two non-equivalent reachable states from the same deal produce different payloads.

Sketch:

1. Different foundation tops ⇒ FOUNDATIONS field differs.
2. Same foundations, different hole tops ⇒ FOUNDATIONS field differs (hole-top encoding).
3. Same foundations/hole, different waste pointer ⇒ WASTE-PTR differs.
4. Same headers, different card locations ⇒ some card has a different descriptor:
   - Different parent ⇒ different PARENT_i.
   - One is root, other is not ⇒ ROOT vs. other value.
   - One is in starting position, other has moved ⇒ STARTING vs. other.
   - One is in cell, other is not ⇒ IN_CELL vs. other.
5. Two states differing only in which root is above which face-down block: this is a pile permutation ⇒ states are equivalent ⇒ same payload (correct).

A formal proof requires case analysis per game type. Empirical validation (solver with both old and new caches, verifying identical results) is recommended.

12 Worst-Case Entry Sizes

All games use the same 32-byte layout. The effective information content varies:

Game	Build pol.	Active descriptor values	Info bits
FreeCell	RED_BLACK	0–5 (6 of 16)	$52 \times 2.58 \approx 134$
Klondike	RED_BLACK	0–5	≈ 134
Baker's Game	SAME_SUIT	0–4 (5 of 16)	$52 \times 2.32 \approx 121$
Fan	SAME_SUIT	0–4	≈ 121
Beleaguered Castle	ANY_SUIT	0–7 (8 of 16)	$52 \times 3 = 156$
Black Hole	NO_BUILD	0–1 (2 of 16)	$52 \times 1 = 52$

All well within the 208-bit descriptor budget. The 4-bit-per-card layout wastes bits for simpler games but maintains uniformity.

13 Future Extensions

Two-deck games. 104 cards × 4 bits = 416 bits for descriptors alone, exceeding 32 bytes. Options: 64-byte entries (one per cache line), or a separate layout. The 4-bit vocabulary accommodates up to 8 parents (PARENT_0–PARENT_7 using values 4–11), sufficient for ANY_SUIT two-deck (8 possible parents per card). Falls back to old cache until implemented.

Suit symmetry streamliners. Under RED_BLACK suit symmetry, suits reduce to colours. Card IDs and parent ordering must be redefined over equivalence classes. The 4-bit descriptor vocabulary is sufficient; only the ID mapping and parent tables change.

Sequences and accordion. Position-indexed games need a fundamentally different encoding (card identity per position). Likely require a separate payload format or fallback to old cache.

Arithmetic coding. For games where descriptor values used $< 2^4$, information-theoretic savings of 1–3 bytes are possible via arithmetic or block coding. Trades encoding complexity for space; unlikely to be worthwhile given the 32-byte budget is already met.

Games with `max_rank` < 13. Fewer than 52 cards; unused descriptor nibbles are zero-filled. No other change.

Games with `foundations_removable`. Cards can leave foundations. Incremental update: on removal, update foundation top and set the card's descriptor to its new tableau value. The encoding handles this naturally.

Games with random `foundations_base`. Foundation encoding is unchanged (4 top ranks); modular rank arithmetic applies when testing "is card on foundations."

References

[1] Zobrist, A. L. (1970). *A New Hashing Method with Application for Game Playing*. Tech. Report 88, CS Dept., University of Wisconsin–Madison.

[2] Clarke, D., Devadas, S., van Dijk, M., Gassend, B., & Suh, G. E. (2003). Incremental Multiset Hash Functions and Their Application to Memory Integrity Checking. *ASIACRYPT 2003*, LNCS 2894, pp. 188–207.

[3] Kun, J. (2021). Group Actions and Hashing Unordered Multisets. https://www.jeremykun.com/2021/10/14/group-actions-and-hashing-unordered-multisets/

[4] Breuker, D. M., Uiterwijk, J. W. H. M., & van den Herik, H. J. (1994). Replacement Schemes for Transposition Tables. *ICCA Journal*, 17(4), 183–193.

[5] Breuker, D. M. (1998). *Memory versus Search in Games*. Ph.D. Thesis, Universiteit Maastricht.

[6] Akagi, Y., Kishimoto, A., & Fukunaga, A. (2010). On Transposition Tables for Single-Agent Search and Planning. *SoCS-10*; extended: *AAAI*, 35(14), 2021.

[7] Slate, D. J. & Atkin, L. R. (1977). CHESS 4.5—The Northwestern University Chess Program. In *Chess Skill in Man and Machine*, pp. 82–118. Springer.

[8] DeepWiki (2025). Transposition Table: official-stockfish/Stockfish. https://deepwiki.com/official-stockfish/Stockfish/4.3-move-ordering-and-selection

[9] Vondele, S. F. (2018). Allow for general transposition table sizes. PR #1341, official-stockfish/Stockfish.

[10] Abseil Team, Google (2018). Swiss Tables Design Notes. https://abseil.io/about/design/swisstables

[11] de Bondt, M. (2003). Compact Freecell Positions. In `fc-solve` documentation.

[12] Blake, C. & Gent, I. P. (2026). The Winnability of Klondike Solitaire and Many Other Patience Games. *JAIR*, 85, Article 21.

[13] Gent, I. P. & AI Assistant (2026). Collaborative Architectures for Zero-False-Positive Caching in Solvitaire. Unpublished working document.